CS 2340 Project Report - Amna Ashraf

Description of Program:

This program simulates a game called “black hole” written in MIPS assembly and runs on MARS. The game includes user input from the I/O console and graphics using the bitmap display tool. The program supports one user vs a computer, where the computer chooses random circles during its turn. The program was designed to be modular, consisting of multiple files that contain procedures to handle different tasks. For example, Graphics.asm contains functions to draw to the board while DrawFunctions.asm contains functions to display specific shapes and characters (e.g. numbers, letters, circles).

Game rules can be read below:

The board consists of 21 circles arranged in a pyramid. Two players (in this case, the user and computer) take turns placing the numbers 1 - 10 in a chosen circle until only one circle remains. The remaining circle is the “black hole” and a player’s score is determined by the sum of their adjacent circles to the black hole. The player with the lowest score wins. Video explanation:https://www.youtube.com/shorts/nrrMWiV1cpQ

Challenges:

- **Learning bitmap display:** The most challenging task by far was learning and implementing the bitmap display. The bitmap display was not covered in class, meaning I had to self learn by watching YouTube tutorials. Initially I had difficulty understanding how the bitmap was represented, but once I understood it was simply a large 2d array and all I needed to do to draw a pixel was to edit the corresponding address, it made much more sense.
- **Implementing custom graphics:** As a low level language, assembly causes tasks to require significantly more planning and code compared to higher level language. I underestimated how difficult drawing simple numbers and letters would be. At the beginning I was planning on manually drawing every single pixel for the numbers and letters, but I realized that would consume an insane amount of time and require many lines of code. Instead, I made functions that would draw a pixel or horizontal/vertical lines given an x,y coordinate, color value, and line length if applicable. These functions made drawing the numbers and letters much faster as I no longer needed to draw every pixel by hand, I could easily draw a line with one function call. I honestly felt like I was developing a drawing program rather than a game as I kept adding layers of abstraction (draw pixel -> draw line -> draw shape/numbers/letters).
- **Writing modular code:** I struggled with connecting my files together. I thought I could simply .include them at the top of the main file and didn’t realize the placement of the .include is relevant. After some time, I realized that I needed to run my main function first to set up the registers correctly before including the other files.
- **Memory management:** I had difficulty keeping track of which registers I reserved for certain values and I would accidentally override them. I fixed this by clearly stating what

each function uses as parameters and what registers they return in a function header and keeping a list of registers with a designated purpose.

- **Return addresses:** My code includes several layers of jal-ing to other functions, meaning several return addresses to keep track of. At some point the return addresses weren't being restored correctly and it was tedious going through layers of jals. Eventually, I realized that I was using beq the same way as a jal, which caused issues because beq does not save the previous jump's return address.
- **Poor planning:** When choosing the size of my display and letters, I did not plan properly, resulting in hours of lost time. I wanted to write "Player wins" or "Computer wins" at the end of the game, so I painstakingly wrote functions to draw each letter in those phrases. It took time even with my draw line functions and resulted in over 1k lines of code. At the end, I realized "computer" was too long by one letter and only "compute" would fit on the screen. So, I instead displayed "You win" or "You lose", which uses only 9/15 of the letters I implemented, wasting hours and over 300 lines of code.

What I Learned:

This project allowed me to become confident in writing and understanding assembly code. I learned how to plan and break a large project into smaller tasks, considering memory management, code flow, and code efficiency. I was forced to think more deeply about the project structure compared to when I designed projects that used high level languages. I learned how graphics are implemented and how to navigate balancing game logic and graphics functions, in addition to designing pixel art. I also learned time management, as I was forced to work on this project multiple days in advance if I wanted to complete it on time.

Discussion of Algorithms:

- **Graphics:**
  - **Making core drawing functions:**
    - **calculateAddress:** To display pixels on the map, you need to write to the correct memory address. I wanted to be able to pass in an x,y coordinate (map is 64x64 units) to future functions instead of a memory address to make referencing positions easier. So, I created a function to convert x,y coordinates by multiplying x by 4 and y by 256 then adding it to the base address and finally returning the updated memory address
    - **getColor:** A hex color code must be written to the corresponding memory address to draw a colored pixel. I wanted to easily switch between colors by passing a color to $a2 and then call a draw function to use that color. To do this, I made a ColorTable array in memory that holds the hex code of different colors. $a2 would reference the index in ColorTable (ex. 0 = white). The getColor function returns the hex value of the color by accessing the correct index of ColorTable. It basically does colorTable[index].
    - **drawPixel**: I developed the drawPixel function, which draws a pixel given an x,y coordinate and a hex color code. The drawPixel function first calls the calculateAddress function to get the correct memory address, then

getColor to get the correct hex color. Finally, it stores the hex color to the memory address to display the pixel.
    - **drawHorizontalLine:** I didn't want to repeatedly draw one pixel at a time, so I made a draw line function. This function takes an x,y coordinate as the start of the line (draws from left to right), a color, and length of line. The function starts at the given x coordinate and loops line length number of times, incrementing the x coordinate by 1 to draw the next right pixel. \
    - **drawVerticalLine:** Same process as drawHorizontalLine but draws a line from top to bottom, incrementing the y coordinate.
- **Displaying the board:** Drawing the background uses a nested loop that iterates over the entire board and calls the drawPixel function, which draws a pixel at a specific x,y coordinate with the chosen color.
- **Drawing the circles:** I developed a function that draws a circle given an x,y coordinate as the center. I made 2 arrays that stored the x,y positions of the center of all 21 circles, and then looped through those arrays and called the drawCircle function to draw a circle at those coordinates. I used trial and error to finalize the coordinates for the circles.
- **Displaying the numbers:** DrawFunctions.asm is a module that contains all of the hardcoded graphics. I made functions to draw the numbers 1-10 given a center x,y coordinate. I used a combination of my drawPixel and drawHorizontal/Vertical line functions to draw the corresponding number based on the center x,y coordinate. When the user inputs their circle choice, access the corresponding xy coordinates from the x/yCircleCenters and pass those to the corresponding number draw function to draw the number inside the circle.
- **Displaying "You win" or "You lose":** Similarly to the numbers, I made functions that draw a letter given a color and center x,y coordinates. I made two other functions, drawPlayerWinner and drawPlayerLose that call these letter functions to display the corresponding message on screen.

- **Game logic:**
    - **Game loop:** When the program starts, the board is drawn. A counter for available circles and the current number to be placed is initialized. While there are circles remaining, the program loops and processes the player's turn, then the computer's turn. The available circles decrease by one while the current number increases by 1. Once there are no available circles, the scores are calculated. If the player's score is less than the computer's draw "You win" on the screen. If not, draw "You lose" on the screen. Finally, exit the program.
    - **Processing player turn:**
        - **Getting and validating input:** First, prompt the player to enter their chosen index from 1-21. Read the integer, save and subtract by one to get the correct 0 index number, and copy to a temp variable. Check if the saved input is valid by checking if it's less than 0 or greater than 20.Then, load "circleValues", the array that stores the values of the circles, calculate the address offset by multiplying the temporary and adding it to the loaded array address. Then, check if the circle is occupied by

checking if circleValues[index] != 0. If it is, repeat the process of prompting and saving input until a valid answer is given. Once input is validated, move to the next part.

- **Updating game based on input:** Since circleValue's address is already loaded, save the user's input at circleValue[index]. Load the circleTeams array, update the address to point to the correct index, then save "1" at the chosen circle. This signifies that this circle is owned by the player and will be used when calculating scores.Finally, call the "drawNumberOnCircle" function, which draws the current number on the chosen circle.

- **Processing computer turn:**
  - **Choose a random circle and validate:** Choose a random number (0-20) using SysRandIntRange service. Load the circleValeus array and check if the circle at the chosen index is taken. If it is, choose another random number until the circle is not occupied.
  - **Update game based on circle value:** Uses the same algorithm from player turn to update game, except save a 0 instead of 1 for the team value.
- **Calculating scores:**
  - There's a whole module dedicated to calculating scores (HandleWinner.asm). The calculateScore function first calls the findBlackHole function, which loops through circleValues until it finds "0" (where the black hole is) and saves the index. Then, load the pyramidPosition array, which stores the position a circle is in relation to the pyramid (top, corner, center, bottom, etc) and get the corresponding position of the black hole. Depending on the position, call the corresponding function to get the adjacent circles (for example, if the black hole is on the bottom, get the left, top left, top right, and right circles). When checking adjacent circles, check what team the circle is, and add the circle values to the corresponding player ($v0 for user, $v1 for computer).
    - **Finding adjacent circles:** To get a specified adjacent circle, you can add or subtract the blackhole's index and the number of circles of the black hole's layer. For example, to get the bottom right circle index, add the black hole index + layer width + 1. To avoid getting circles out of bounds (for example, attempting to get the bottom left circle when the black hole is on the bottom), I coded functions that hardcoded what circles to access depending on the position.
- **Finding scores and exiting program:**
  - After calculating the scores, they are stored in $v0 and $v1. Back in main, check if the player's score is less than the computer's ($v0 < $v1). If it is, set the brush color to blue and call the drawPlayerWinner function to draw

“You win” on the screen. If not, set the brush color to red and draw “You lose” on the screen.